SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Kamali (192524423) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kamali (192524423)

Question 1: Hospital Shift Scheduling by Backtracking Search

1.1 Problem Understanding and CSP Formulation

This is a Constraint Satisfaction Problem (CSP). Three doctors must each receive exactly one of three shifts, and no shift may be shared.

- **Variables:** D1, D2, D3 (the doctors).
- **Domain of each variable:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N.
- **Constraints:** C1: D1 ≠ N. C2: shift(D2) < shift(D3). C3: all three shifts are different (one doctor per shift; with 3 doctors and 3 shifts this also guarantees that every doctor gets exactly one shift). C4: D3 ≠ M.

Unary constraints applied first (node consistency): C1 reduces the domain of D1 to {M, A}, and C4 reduces the domain of D3 to {A, N}. D2 keeps {M, A, N}.

1.2 Backtracking Search with Constraint Checks

Variables are assigned in the order D1, D2, D3 and values are tried in the order M, A, N. After every assignment, all constraints whose variables are already assigned are checked; a failure triggers backtracking to the next value.

Step	Assignment	Constraint check	Result
1	D1 = M	C1: M ≠ N ✓. No other variable assigned yet.	Accept
2	D1 = M, D2 = M	C3: D2 shares a shift with D1 ✗	Reject, try next
3	D1 = M, D2 = A	C3 ✓ (M ≠ A). C2 cannot be tested yet (D3 unassigned).	Accept
4	D1 = M, D2 = A, D3 = A (first value in D3's domain)	C3: A is already used by D2 ✗	Reject, try next
5	D1 = M, D2 = A, D3 = N	C1 ✓. C2: A < N ✓. C3: M, A, N all different ✓. C4: N ≠ M ✓.	**All satisfied: solution**

No further backtracking was needed, because the first complete assignment satisfied every constraint. (Node consistency removed M from D3's domain in advance, so the useless branch D3 = M was never tried.)

1.3 Final Valid Schedule

Morning	Afternoon	Night
D1	D2	D3

Interpretation: D1 avoids the night shift, D2 (Afternoon) works before D3 (Night), D3 is not on Morning duty, each shift has exactly one doctor, and each doctor has exactly one shift. If the search is continued after the first solution, backtracking finds one more valid schedule: D1 = Afternoon, D2 = Morning, D3 = Night. All other permutations violate at least one constraint, so there are exactly two solutions and the schedule above is the first one found. Backtracking is efficient here because early constraint checks cut off invalid branches before a full assignment is built.

Question 2: Grid Navigation with BFS (5×5)

2.1 Problem Understanding and Assumptions

The robot moves on a 5×5 grid from S to G using Up, Down, Left and Right moves, each costing 1, and cannot enter obstacle cells. Cells are written (row, column) with rows 0–4 from top to bottom and columns 0–4 from left to right. The grid figure was not supplied with the question, so the following layout is assumed: **S = (0,0), G = (4,4), obstacles (■) = (1,1), (1,3), (3,1), (3,3).** The same method applies to any other layout.

S	·	·	·	·
·	■	·	■	·
·	·	·	·	·
·	■	·	■	·
·	·	·	·	G

2.2 Method Selection

BFS expands nodes level by level using a FIFO queue. Because every move costs 1, BFS is **complete and optimal**: the first time it reaches G, it has used the fewest moves, which equals the least cost. Neighbours are generated in the order Up, Down, Left, Right, and visited cells are never added again. Note that BFS itself does not use a heuristic (ordering the queue by Manhattan distance would make it Greedy or A* search). The Manhattan distance $h(n) = |r - 4| + |c - 4|$ is therefore used afterwards to verify that the path found is optimal.

2.3 Step-by-Step Node Expansion and Queue Updates

Step	Expand (depth)	New nodes added to queue	Queue after step (front → back)
1	(0,0) d0	(1,0), (0,1)	(1,0), (0,1)
2	(1,0) d1	(2,0)	(0,1), (2,0)
3	(0,1) d1	(0,2)	(2,0), (0,2)
4	(2,0) d2	(3,0), (2,1)	(0,2), (3,0), (2,1)
5	(0,2) d2	(1,2), (0,3)	(3,0), (2,1), (1,2), (0,3)
6	(3,0) d3	(4,0)	(2,1), (1,2), (0,3), (4,0)
7	(2,1) d3	(2,2)	(1,2), (0,3), (4,0), (2,2)
8	(1,2) d3	none (neighbours are obstacles or already queued)	(0,3), (4,0), (2,2)
9	(0,3) d3	(0,4)	(4,0), (2,2), (0,4)
10	(4,0) d4	(4,1)	(2,2), (0,4), (4,1)
11	(2,2) d4	(3,2), (2,3)	(0,4), (4,1), (3,2), (2,3)
12	(0,4) d4	(1,4)	(4,1), (3,2), (2,3), (1,4)
13	(4,1) d5	(4,2)	(3,2), (2,3), (1,4), (4,2)
14	(3,2) d5	none	(2,3), (1,4), (4,2)
15	(2,3) d5	(2,4)	(1,4), (4,2), (2,4)
16	(1,4) d5	none	(4,2), (2,4)
17	(4,2) d6	(4,3)	(2,4), (4,3)
18	(2,4) d6	(3,4)	(4,3), (3,4)

Step	Expand (depth)	New nodes added to queue	Queue after step (front → back)
19	(4,3) d7	**(4,4) = G** (depth 8)	(3,4), (4,4)
20	(3,4) d7	none	(4,4)
21	(4,4) d8	Goal test succeeds on removal from the queue	**Goal found**

2.4 Optimal Path and Cost

Following the parent pointers back from G gives the path **(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)**, that is, 4 moves Down followed by 4 moves Right. **Path cost = 8 × 1 = 8.**

Verification with the Manhattan heuristic: h(S) = |0−4| + |0−4| = 8, and the path cost is also 8. Manhattan distance never overestimates the true cost on a 4-connected grid (it is admissible), so no path can be cheaper than 8, which confirms optimality. Every node on the path has g + h = 8 (for example (2,0): g = 2, h = 6; (4,2): g = 6, h = 2). An A* search, which orders its priority queue by f = g + h, would therefore expand mostly nodes with f = 8 and reach the same answer with fewer expansions. Several equal-cost paths exist, and BFS returns the first one found, which depends on the neighbour order.

Interpretation: the robot needs 8 moves, which is the theoretical minimum, so the obstacles force no detour in this layout. BFS processed 21 cells (every free cell), which shows why heuristic search is preferred on larger grids.

Question 3: Autonomous Rescue Robot (Online Search)

(a) Analysis of Constraints and Their Effect on Decision-Making

Constraint	Effect on the robot's decisions
Partial observability (senses only adjacent cells)	The robot has no complete map, so it cannot plan the whole route offline. It must act, sense, update its map and re-plan. This is the most influential constraint because it makes the task an online search problem.
Blocked cells (obstacles)	Some moves are illegal, and a wall may be discovered only when the robot is adjacent to it. Planned paths can become invalid, forcing re-planning and sometimes backtracking from dead ends.
Move cost 1; entering a risky zone costs 1 + 2 = 3	The cost is non-uniform, so the fewest-steps path is not necessarily the cheapest. The robot may prefer a longer but safe detour over a short route through risky cells.
Minimise total cost with safe navigation	This is an optimisation objective with a safety aspect. Risk is penalised through cost, so the robot avoids risky cells unless the detour is more expensive than the risk.
Dynamic knowledge update	The robot keeps a belief map (cells marked free, blocked, risky or unknown) and updates it after each move. Decisions are based on current knowledge, not on the final truth.

Prioritisation: (1) obstacle avoidance and safety (hard constraints), (2) cost minimisation, (3) exploration efficiency. Unknown cells are treated optimistically as free with cost 1 (the free-space assumption) and corrected as soon as sensing shows otherwise.

(b) Solution Approach: Online Uniform Cost Search with Re-planning

The proposed approach is an **online search agent that runs Uniform Cost Search (UCS) on its current map and re-plans after each new observation.** UCS suits the problem because step costs differ (1 or 3) and UCS always returns the least-cost path on the known map. Adding the Manhattan-distance heuristic turns it into A*, which stays optimal (the heuristic is admissible because every step costs at least 1) and expands fewer nodes.

Steps involved:

1. **Initialise:** the map contains S, G and the surroundings of S; all other cells are 'unknown'. Current position = S.
2. **Sense:** observe the four adjacent cells and record each as free, blocked or risky.
3. **Update the map:** add the new facts; if the current plan contains a blocked cell, discard the plan.
4. **Plan:** run UCS (or A*) from the current cell to G over cells not known to be blocked, with cost 1 for free and unknown cells and 3 for risky cells.
5. **Act:** execute only the first move of the plan, move to the new cell and add its cost to the total.
6. **Repeat** steps 2–5 until G is reached. If no path exists on the known map, explore the cheapest unexplored frontier cell; if nothing remains, report failure (survivor unreachable).

```
function ONLINE-RESCUE-AGENT(percept) returns an action
    persistent: map, pos, goal G, totalCost ← 0
    UPDATE-MAP(map, percept)        // adjacent cells: free / blocked / risky
    if pos = G then return STOP
    path ← UCS(map, pos, G)         // cost 1, risky entry 3, unknown 1
    if path = failure then
        path ← PATH-TO-NEAREST-UNEXPLORED-CELL(map, pos)
        if path = failure then return STOP   // survivor unreachable
    move ← FIRST-STEP(path)
    totalCost ← totalCost + STEP-COST(move)
    pos ← RESULT(pos, move)
    return move
```

Illustration: suppose the apparent shortest route passes through a cell that the robot later senses as blocked. It updates the map, UCS recomputes, and the robot goes around. If the choice is a risky cell (cost 3) versus a safe detour adding 2 extra cells (cost 2), the agent takes the detour; if the detour would cost 4, it accepts the risky cell.

(c) Application of AI Concepts

Concept	Application to the rescue robot
Intelligent agent	The robot perceives its environment through sensors (adjacent-cell sensing, position) and acts through actuators (moving up, down, left or right) in a perceive → update → plan → act loop. It is a goal-based agent (goal: reach G) with a utility aspect (minimise cost and risk) and an internal model (the map), so it is also model-based.
Rationality	A rational agent picks the action expected to maximise its performance measure given its percept history and knowledge. Here the measure is minimum total cost with safe navigation. The robot is rational because it chooses the cheapest known route, avoids risky cells unless justified by cost, and updates its beliefs instead of repeating failed moves. Rationality is not omniscience, so a detour caused by an undiscovered wall is not irrational.
Environment type	Partially observable (only adjacent cells), single agent, deterministic (a move succeeds unless blocked), sequential (earlier moves affect later costs), static (the building is assumed not to change while the robot moves), discrete (grid cells), and unknown (the map is learned online).

(d) Justification of the Chosen Approach

- **Handles unequal costs:** BFS and DFS ignore costs, whereas UCS/A* minimise total cost, which is the stated objective.
- **Handles partial observability:** an offline planner assumes a complete map. Online search with re-planning works with limited sensing and corrects itself after every observation.
- **Optimal on the known map:** UCS gives the minimum-cost path, and re-planning ensures the robot always follows the current best route around newly discovered obstacles.
- **Supports safe navigation:** risk is encoded as extra cost, so the agent avoids risky zones unless crossing them is substantially cheaper.
- **Efficient in practice:** the grid is small, so repeated UCS/A* calls are cheap, and the Manhattan heuristic reduces expansions further.
- **Compared with alternatives:** greedy or hill-climbing search can get trapped in dead ends, and learning methods such as LRTA* converge slowly, while re-planning UCS/A* is robust and easy to justify.

Conclusion: an online re-planning agent using Uniform Cost Search (or A* with Manhattan distance) best satisfies the limited sensing, obstacle, risk-cost and minimum-cost requirements of the rescue mission.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided